Witle

Academy

WEEK 5 – 9

SQL Fundamentals

for Data Engineering & Analytics

- CASE AND IF
- Window Functions
- Subqueries
- IF
- DML
- DDL
- CTEs
- Intervals

01

CASE Statement

Conditional Logic Inside Your Queries

What is the CASE Statement?

Witle Academy

Conditional Logic

Works like IF/ELSE but inside SQL queries — no external scripting needed.

Returns a Value

Evaluates conditions row by row and returns exactly one matching result.

Used Anywhere

Works in SELECT, ORDER BY, WHERE, GROUP BY, and HAVING clauses.

ANSI Standard

Fully supported by MySQL, PostgreSQL, SQL Server, and Oracle.

Think of CASE like a grade check:

```
CASE score
  WHEN 90 THEN 'A'
  WHEN 80 THEN 'B'
  WHEN 70 THEN 'C'
  ELSE       'F'
END
```

Each row gets a label based on its value.

CASE Statement: Two Forms

Witle Academy

Simple CASE

```
CASE expression
  WHEN value1 THEN result1
  WHEN value2 THEN result2
  ELSE default_result
END
```

Compares ONE expression against a set of fixed values.

Searched CASE

```
CASE
  WHEN condition1 THEN result1
  WHEN condition2 THEN result2
  ELSE default_result
END
```

Evaluates independent boolean conditions for each row.

ELSE is optional — if omitted and no WHEN matches, SQL returns NULL.

Example 1: Simple CASE

```
SELECT
  student_name,
  grade,
  CASE grade
    WHEN 'A' THEN 'Excellent'
    WHEN 'B' THEN 'Good'
    WHEN 'C' THEN 'Average'
    ELSE 'Needs Improvement'
  END AS feedback
FROM students;
```

T betwe. 1 and 10 the

T >1 and <10

Output

student_name	grade	feedback
Alice	A	Excellent
Bob	C	Average
Carol	B	Good
Dan	D	Needs Improvement

CASE grade compares the grade column directly to each WHEN value — first match wins.

Example 2: Searched CASE

```
SELECT
  product_name,
  price,
  CASE
    WHEN price < 10   THEN 'Budget'
    WHEN price < 50   THEN 'Mid-Range'
    WHEN price < 200  THEN 'Premium'
    ELSE                   'Luxury'
  END AS tier
FROM products;
```

(Price - C) > 10

C

Output

product_name	price	tier
Pen	$2	Budget
Notebook	$15	Mid-Range
Laptop Bag	$75	Premium
MacBook Pro	$2499	Luxury

Searched CASE evaluates range conditions (price < 10) rather than exact matches.

Using CASE in ORDER BY & GROUP BY

Witle Academy

ORDER BY

```
SELECT name, status
FROM orders
ORDER BY
  CASE status
    WHEN 'Urgent'  THEN 1
    WHEN 'Normal'  THEN 2
    ELSE                3
  END;
```

Sort rows using custom priority instead of alphabetical order.

GROUP BY

```
SELECT
  CASE
    WHEN age < 18 THEN 'Minor'
    WHEN age < 65 THEN 'Adult'
    ELSE               'Senior'
  END AS age_group,
  COUNT(*) AS total
FROM users
GROUP BY age_group;
```

Create on-the-fly categories for aggregation and counting.

Common Mistakes & Best Practices

Witle Academy

✗ Missing ELSE

Without ELSE, unmatched rows silently return NULL.

✗ Overlapping Conditions

SQL picks the FIRST matching WHEN - order matters!

✗ Type Mismatch

All THEN branches must return compatible data types.

✓ Alias Your CASE

Use AS column_name for a clean, readable result column.

✓ Nest With Care

CASE can nest inside CASE - but keep it readable.

✓ Use in CTEs

Wrap complex CASE logic in a CTE for reusability.

Summary

Witle Academy

CASE **Conditional Logic**

Adds IF/ELSE logic directly inside SQL queries.

2 **Two Forms**

Simple (value match) and Searched (conditions).

4x **Versatile Use**

Works in SELECT, ORDER BY, GROUP BY & WHERE.

NULL **Watch for NULL**

Always add ELSE to avoid unexpected NULLs.

IIF vs CASE — Which Should You Use?

Witle Academy

IIF( )

```
IIF( condition,
  true_value,
  false_value
)

-- Example:
IIF(salary > 5000,
  'High', 'Low'
) AS pay_band
```

YES Quick shorthand for true/false only

YES SQL Server, Access, MySQL 8+

NO Max 2 outcomes - no multi-branch

vs

CASE ... END

```
CASE
  WHEN salary > 8000 THEN 'Senior'
  WHEN salary > 5000 THEN 'Mid'
  ELSE 'Junior'
END AS pay_band
```

YES Unlimited branches and conditions

YES All databases - ANSI standard

YES Works in SELECT, ORDER BY, GROUP BY

Witle Academy

02

Common Table Expressions

Writing clean, readable, and reusable SQL logic

What is a CTE?

Definition

A CTE (Common Table Expression) is a named temporary result set defined with the WITH keyword. It exists only for the duration of the query.

Why use CTEs?

- Improves readability & structure
- Replaces nested subqueries
- Can be referenced multiple times
- Supports recursion

```
WITH cte_name AS (
  SELECT
    employee_id,
    department,
    salary
  FROM employees
  WHERE active = TRUE
)
SELECT *
FROM cte_name
WHERE salary > 50000;
```

Chaining Multiple CTEs

You can chain multiple CTEs using commas — each one can reference the previous.

```
WITH
  dept_avg AS (
    SELECT department, AVG(salary) AS avg_salary
    FROM employees
    GROUP BY department
  ),
  high_earners AS (
    SELECT e.name, e.department, e.salary, d.avg_salary
    FROM employees e
    JOIN dept_avg d USING (department)
    WHERE e.salary > d.avg_salary
  )
SELECT *
FROM high_earners
ORDER BY salary DESC;
```

💡 *Tip: Think of each CTE as a building block — chain them to build complex logic step by step.*

03

Subqueries

Nesting queries to power complex filtering and lookups

Types of Subqueries

Scalar Subquery

Returns a single value. Used in SELECT or WHERE.

```
WHERE salary > (SELECT AVG(salary) FROM employees)
```

Row Subquery

Returns one row with multiple columns.

```
WHERE (dept, level) = (SELECT dept, level FROM ref_table WHERE id = 1)
```

Column Subquery

Returns a list of values. Used with IN / ANY / ALL.

```
WHERE dept_id IN (SELECT id FROM departments WHERE active = TRUE)
```

Correlated Subquery

References the outer query — executes once per row.

```
WHERE salary > (SELECT AVG(salary) FROM e2 WHERE e2.dept = e.dept)
```

04

Window Functions

Perform calculations across related rows without collapsing them

What Are Window Functions?

Window functions perform calculations across a set of rows **related to the current row**, without collapsing rows like GROUP BY does. They're one of the most powerful features in SQL.

What Are Window Functions?

OVER()

Defines the window — which rows to include in the calculation

PARTITION BY

Divides rows into groups (like GROUP BY but keeps all rows)

ORDER BY

Determines the row ordering within each window partition

Frame Clause

Specifies the subset of rows: ROWS BETWEEN ... AND ...

```
Syntax:  function_name()  OVER  (  [PARTITION BY ...]  [ORDER BY ...]  [frame]  )
```

Common Window Functions

ROW_NUMBER()

Unique row number in partition

RANK()

Rank with gaps for ties

DENSE_RANK()

Rank without gaps for ties

SUM() OVER()

Running total in partition (with ORDER BY)

AVG() OVER()

Moving average (with ORDER BY & frame)

LAG() / LEAD()

Access previous / next row value

More Window Functions

NTILE()

Divide rows into N equal buckets

FIRST_VALUE()

First value in the window frame

LAST_VALUE()

Last value in the window frame

NTH_VALUE()

Nth value in the window frame

PERCENT_RANK()

Relative rank as a percentage (0 to 1)

CUME_DIST()

Fraction of rows ≤ current row

SQL EXAMPLE

RANK()

```sql
SELECT
  employee_name,
  department,
  salary,
  RANK() OVER (
    PARTITION BY department
    ORDER BY salary DESC
  ) AS salary_rank
FROM employees;
```

What it does

Assigns a rank to each row based on the ORDER BY value.

Tied values get the same rank, then the next rank is skipped.

e.g. if two rows tie for rank 2, the next rank is 4 (not 3).

Sample output

```
Alice  | Sales | 70k | 1
Bob    | Sales | 60k | 2
Carol  | Sales | 60k | 2
Dan    | Sales | 50k | 4
```

SQL EXAMPLE

DENSE_RANK()

```sql
SELECT
  employee_name,
  department,
  salary,
  DENSE_RANK() OVER (
    PARTITION BY department
    ORDER BY salary DESC
  ) AS dense_rnk
FROM employees;
```

What it does

Like RANK(), but no gaps in the ranking sequence.

Tied values share a rank, and the next distinct value gets the very next integer.

Sample output

```
Alice  | Sales | 70k | 1
Bob    | Sales | 60k | 2
Carol  | Sales | 60k | 2
Dan    | Sales | 50k | 3
```

SQL EXAMPLE

SUM() OVER()

```
SELECT
  month,
  revenue,
  SUM(revenue) OVER (
    ORDER BY month
    ROWS BETWEEN UNBOUNDED
      PRECEDING AND CURRENT ROW
  ) AS running_total
FROM monthly_sales;
```

What it does

Calculates a running (cumulative) sum when used with ORDER BY.

Without ORDER BY, returns the total partition sum on every row.

The frame clause controls which rows are included in the sum.

Sample output

```
Jan | 100 | 100
Feb | 150 | 250
Mar | 120 | 370
Apr | 180 | 550
```

SQL EXAMPLE

AVG() OVER()

```
SELECT
  month,
  revenue,
  AVG(revenue) OVER (
    ORDER BY month
    ROWS BETWEEN 2 PRECEDING
      AND CURRENT ROW
  ) AS moving_avg_3m
FROM monthly_sales;
```

What it does

Computes a moving average over a sliding window of rows.

The frame clause (ROWS BETWEEN) defines the window size.

Here: 3-month moving average (current + 2 preceding).

Sample output

```
Jan | 100 | 100.0
Feb | 150 | 125.0
Mar | 120 | 123.3
Apr | 180 | 150.0
```

SQL EXAMPLE

LAG() / LEAD()

```sql
SELECT
  month,
  revenue,
  LAG(revenue, 1) OVER (
    ORDER BY month
  ) AS prev_month,
  LEAD(revenue, 1) OVER (
    ORDER BY month
  ) AS next_month
FROM monthly_sales;
```

What it does

LAG() accesses a value from a previous row.

LEAD() accesses a value from a following row.

Great for calculating period-over-period changes.

Sample output

```
Jan | 100 | NULL | 150
Feb | 150 | 100  | 120
Mar | 120 | 150  | 180
Apr | 180 | 120  | NULL
```

SQL EXAMPLE

NTILE()

```sql
SELECT
  employee_name,
  salary,
  NTILE(4) OVER (
    ORDER BY salary DESC
  ) AS quartile
FROM employees;
```

What it does

Divides the result set into N roughly equal buckets.

Useful for percentile analysis (quartiles, deciles, etc.).

If rows don't divide evenly, earlier buckets get one extra row.

Sample output

```
Alice | 90k | 1
Bob   | 80k | 1
Carol | 70k | 2
Dan   | 60k | 2
Eve   | 50k | 3
Frank | 40k | 4
```

SQL EXAMPLE

FIRST_VALUE()

```
SELECT
  employee_name,
  department,
  salary,
  FIRST_VALUE(employee_name) OVER (
    PARTITION BY department
    ORDER BY salary DESC
  ) AS top_earner
FROM employees;
```

What it does

Returns the first value in the ordered window frame.

Useful for comparing each row against the top/first entry.

Every row in the partition sees the same first value.

Sample output

```
Alice | Sales | 70k | Alice
Bob   | Sales | 60k | Alice
Carol | Sales | 50k | Alice
```

SQL EXAMPLE

LAST_VALUE()

```
SELECT
  employee_name,
  department,
  salary,
  LAST_VALUE(employee_name) OVER (
    PARTITION BY department
    ORDER BY salary DESC
    ROWS BETWEEN UNBOUNDED
      PRECEDING AND UNBOUNDED
      FOLLOWING
  ) AS lowest_earner
FROM employees;
```

What it does

Returns the last value in the window frame.

Important: you must set the frame to UNBOUNDED FOLLOWING, otherwise the default frame ends at the current row.

A common gotcha for beginners!

Sample output

```
Alice | Sales | 70k | Carol
Bob   | Sales | 60k | Carol
Carol | Sales | 50k | Carol
```

SQL EXAMPLE

NTH_VALUE()

```
SELECT
  employee_name,
  department,
  salary,
  NTH_VALUE(employee_name, 2) OVER (
    PARTITION BY department
    ORDER BY salary DESC
    ROWS BETWEEN UNBOUNDED
      PRECEDING AND UNBOUNDED
      FOLLOWING
  ) AS second_highest
FROM employees;
```

What it does

Returns the value at the Nth row in the window frame.

Here N=2 gets the second-highest earner per department.

Returns NULL if the frame has fewer than N rows.

Sample output

```
Alice | Sales | 70k | Bob
Bob   | Sales | 60k | Bob
Carol | Sales | 50k | Bob
```

SQL EXAMPLE

PERCENT_RANK()

```sql
SELECT
  employee_name,
  salary,
  PERCENT_RANK() OVER (
    ORDER BY salary
  ) AS pct_rank
FROM employees;
```

What it does

Returns the relative rank as a value between 0 and 1.

Formula: (rank - 1) / (total_rows - 1).

The first row is always 0, the last is always 1.

Sample output

```
Dan   | 40k | 0.00
Carol | 50k | 0.33
Bob   | 60k | 0.67
Alice | 70k | 1.00
```

SQL EXAMPLE

CUME_DIST()

```sql
SELECT
  employee_name,
  salary,
  CUME_DIST() OVER (
    ORDER BY salary
  ) AS cum_dist
FROM employees;
```

What it does

Returns the cumulative distribution of a value.

Formula: (rows with value <= current) / total_rows.

Always between 0 (exclusive) and 1 (inclusive).

The last row is always 1.

Sample output

```
Dan   | 40k | 0.25
Carol | 50k | 0.50
Bob   | 60k | 0.75
Alice | 70k | 1.00
```

05

UNION OPERATIONS

Combining result sets from multiple queries

UNION Operations

UNION

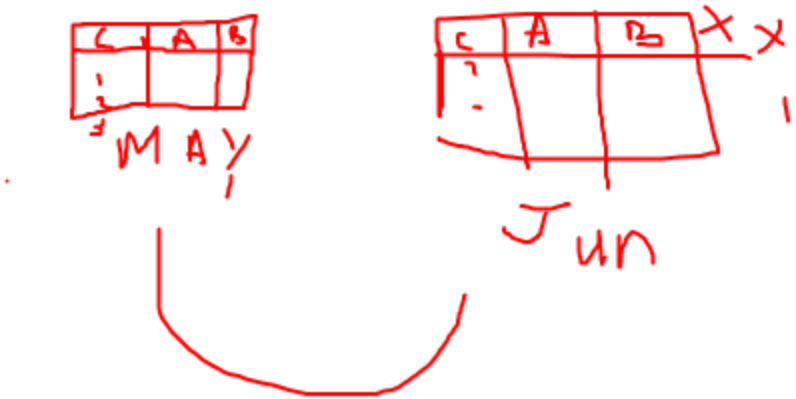
Combine results, remove duplicates

UNION ALL

Combine results, keep all rows

Key Rules

Same column count and compatible types



SQL EXAMPLE

UNION

```
SELECT city FROM customers
UNION
SELECT city FROM suppliers
ORDER BY city;
```

What it does

Combines results from two SELECT statements into one result set.

Automatically removes duplicate rows.

Both queries must return the same number of columns with compatible types.

Sample output

```
Berlin
London
Madrid
Paris
(duplicates removed)
```

SQL EXAMPLE

UNION ALL

```
SELECT city FROM customers
UNION ALL
SELECT city FROM suppliers
ORDER BY city;
```

Select *
From ...
U
...

What it does

Combines results from two queries but keeps ALL rows, including duplicates.

Faster than UNION because it skips the deduplication step.

Use when you know there are no duplicates or you want them.

Sample output

```
Berlin
London
London   (duplicate kept)
Madrid
Paris
Paris    (duplicate kept)
```

COMPARISON

UNION vs UNION ALL

```
-- UNION: 4 rows (deduplicated)
SELECT name FROM table_a
UNION
SELECT name FROM table_b;

-- UNION ALL: 6 rows (all kept)
SELECT name FROM table_a
UNION ALL
SELECT name FROM table_b;
```

Key Differences

UNION removes duplicates (uses DISTINCT internally).

UNION ALL keeps every row — faster performance.

UNION requires a sort/hash to deduplicate.

UNION ALL just appends results.

When to use which

UNION: need unique results (e.g. distinct cities).

UNION ALL: merging logs, combining partitioned tables, or when you know rows are already unique.

06

Intervals

Working with date and time arithmetic in SQL

INTERVALS

Date & Time Arithmetic

Add 7 days

```
date_col + INTERVAL '7 days'
```

Subtract 3 months

```
date_col - INTERVAL '3 months'
```

Add 2 hours 30 mins

```
ts_col + INTERVAL '2 hours 30 minutes'
```

Days between dates

```
end_date - start_date
```

Rows in last 30 days

```
WHERE created_at >= NOW() - INTERVAL '30 days'
```

Truncate to month

```
DATE_TRUNC('month', created_at)
```

INTERVAL Queries in Practice

Add 7 Days

```
SELECT order_id,
       order_date,
       order_date
       + INTERVAL '7 days'
         AS delivery_deadline
FROM   orders;
```

Subtract 3 Months

```
SELECT customer_id, name
FROM   customers
WHERE  signup_date >=
  CURRENT_DATE
  - INTERVAL '3 months';
```

Add 2 Hours 30 Minutes

```
SELECT event_id,start_time,
       start_time
       + INTERVAL '2 hours
         30 minutes'
         AS end_time
FROM   events;
```

Days Between Dates

```
SELECT order_id, ship_date - order_date
         AS shipping_interval,
       EXTRACT(DAY FROM
         ship_date - order_date
       ) AS days_int
FROM   orders;
```

Filter Last 30 Days

```
SELECT COUNT(*)
         AS recent_logins
FROM   user_logins
WHERE  login_at >=
  NOW()
  - INTERVAL '30 days';
```

Truncate to Month

```
SELECT DATE_TRUNC(
         'month', created_at
       ) AS month,
       SUM(revenue) AS total
FROM   sales
GROUP BY 1 ORDER BY 1;
```

07

DDL : DATA DEFINITION

Creating, altering, and dropping database objects

DDL

Data Definition Language

CREATE TABLE

Define a new table with columns and types

ALTER TABLE

Add, modify, or drop columns in a table

DROP TABLE

Permanently delete a table and all its data

TRUNCATE TABLE

Remove all rows without logging each delete

DDL EXAMPLE

CREATE TABLE

```
CREATE TABLE employees (
  id        INT PRIMARY KEY,
  name      VARCHAR(100) NOT NULL,
  email     VARCHAR(255) UNIQUE,
  dept_id   INT,
  salary    DECIMAL(10,2) DEFAULT 0,
  hire_date DATE NOT NULL,
  FOREIGN KEY (dept_id)
    REFERENCES departments(id)
);
```

What it does

Defines a new table with column names, data types, and constraints.

The table is empty after creation — use INSERT to add rows.

Always define a PRIMARY KEY for every table.

Key constraints

```
PRIMARY KEY  — unique row ID
NOT NULL     — required field
UNIQUE       — no duplicates
DEFAULT      — fallback value
FOREIGN KEY  — link to another table
```

DDL EXAMPLE

ALTER TABLE

```
-- Add a new column
ALTER TABLE employees
ADD phone VARCHAR(20);

-- Change column type
ALTER TABLE employees
ALTER COLUMN phone TYPE TEXT;

-- Drop a column
ALTER TABLE employees
DROP COLUMN phone;

-- Rename a column
ALTER TABLE employees
RENAME COLUMN name TO full_name;
```

What it does

Modifies the structure of an existing table without dropping it.

Existing data is preserved (unless you drop a column).

Use with care in production — some changes lock the table.

Common operations

```
ADD        — new column
ALTER      — change type/default
DROP       — remove column
RENAME     — rename column
ADD CONSTRAINT
```

DDL EXAMPLE

DROP TABLE / TRUNCATE

```
-- Drop table (removes everything)
DROP TABLE IF EXISTS temp_logs;

-- Drop with cascade (also drops
-- dependent views/constraints)
DROP TABLE orders CASCADE;

-- Truncate (keep structure,
-- remove all rows)
TRUNCATE TABLE audit_log;
```

What it does

DROP removes the table, its data, indexes, and constraints permanently.

IF EXISTS prevents errors if the table doesn't exist.

CASCADE also drops dependent objects (views, FKs).

DROP vs TRUNCATE

```
DROP    = delete table + data
TRUNCATE = delete data only
TRUNCATE is faster (no row-by-row
logging)
TRUNCATE resets auto-increment
TRUNCATE cannot be rolled back in most
DBs
```

08

DML : DATA MANIPULATION

Inserting, updating, and deleting data in tables

DML

Data Manipulation Language

INSERT INTO

Add new rows to a table

UPDATE

Modify existing rows in a table

DELETE

Remove rows from a table

SELECT

Query and retrieve data from tables

MERGE / UPSERT

Insert or update in a single statement

INSERT ... SELECT

Insert rows from another query result

DML EXAMPLE

INSERT INTO

```
-- Single row insert
INSERT INTO employees
  (name, email, dept_id, salary)
VALUES
  ('Alice', 'alice@co.com', 1, 75000);

-- Multi-row insert
INSERT INTO employees
  (name, email, dept_id, salary)
VALUES
  ('Bob', 'bob@co.com', 2, 65000),
  ('Carol', 'carol@co.com', 1, 70000);
```

What it does

Adds one or more new rows to a table.

Column list is optional if you provide values for ALL columns in order.

Always specify column names for clarity and safety.

Variations

```
Single row  — VALUES (...)
Multi-row   — VALUES (...),(..)
From query  — INSERT...SELECT
With default — omit column
```

DML EXAMPLE

UPDATE

```
-- Update specific rows
UPDATE employees
SET salary = salary * 1.10
WHERE department = 'Engineering';

-- Update multiple columns
UPDATE employees
SET
  salary = 80000,
  dept_id = 3
WHERE id = 42;
```

What it does

Modifies existing data in one or more rows.

The SET clause specifies which columns to change.

The WHERE clause filters which rows are affected.

Safety tip

```
ALWAYS use a WHERE clause!
Without WHERE, UPDATE changes
EVERY row in the table.
Test with SELECT first:
SELECT * FROM employees
WHERE department = 'Engineering';
```

DML EXAMPLE

DELETE

```sql
-- Delete specific rows
DELETE FROM employees
WHERE hire_date < '2020-01-01';

-- Delete with subquery
DELETE FROM orders
WHERE customer_id IN (
  SELECT id FROM customers
  WHERE status = 'inactive'
);

-- Delete all rows (use carefully!)
DELETE FROM temp_logs;
```

What it does

Removes rows from a table based on a condition.

Always use WHERE to target specific rows.

Without WHERE, it deletes ALL rows (but keeps the table structure).

DELETE vs TRUNCATE

DELETE logs each row removal

DELETE can be rolled back

DELETE fires triggers

TRUNCATE is faster for all rows

TRUNCATE cannot be rolled back

09

Data Conversions

CAST, CONVERT, TO_CHAR and more — transforming data types in SQL

CAST() & CONVERT()

CAST()

Converts an expression to a target data type. Standard ANSI SQL — works across all major platforms.

SYNTAX

```
CAST(expression AS data_type)
```

EXAMPLES

```
-- String to Integer
SELECT CAST('42' AS INTEGER);
-- Float to Integer
SELECT CAST(price AS INT)
  FROM products;
-- Date to String
SELECT CAST(order_date
  AS VARCHAR) FROM orders;
```

✓ SQL Server ✓ PostgreSQL ✓ MySQL ✓ Oracle

CONVERT()

Similar to CAST() but includes an optional style parameter for formatting, especially dates/times.

SYNTAX

```
CONVERT(data_type, expression
       [, style])
```

EXAMPLES

```
-- Date to string (style 103)
SELECT CONVERT(VARCHAR,
  GETDATE(), 103); -- dd/mm/yy
-- String to decimal
SELECT CONVERT(DECIMAL(10,2),
  '99.95');
-- ISO format (style 112)
SELECT CONVERT(VARCHAR,
  order_date, 112) FROM orders;
```

✓ SQL Server ✓ MySQL (syntax varies)

TO_CHAR / TO_DATE / TO_NUMBER

TO_CHAR()

Converts a number or date to a formatted string using a format model.

`TO_CHAR(expr, 'format_model')`

✓ Oracle ✓ PostgreSQL

```
-- Date to formatted string
SELECT TO_CHAR(order_date,
  'DD-Mon-YYYY') FROM orders;
-- Number with commas
SELECT TO_CHAR(salary,
  '999,999.00') FROM emp;
```

TO_DATE()

Converts a string to a DATE value using a format model.

`TO_DATE(string, 'format_model')`

✓ Oracle ✓ PostgreSQL

```
-- Parse a date string
SELECT TO_DATE('2025-03-15',
  'YYYY-MM-DD');
-- Non-standard format
SELECT TO_DATE('15/03/2025',
  'DD/MM/YYYY');
```

TO_NUMBER()

Converts a string to a numeric value using a format model.

`TO_NUMBER(string, 'format_model')`

✓ Oracle ✓ PostgreSQL (limited)

```
-- Currency string to number
SELECT TO_NUMBER('1,234.56',
  '9,999.99'); -- 1234.56
-- In a column context
SELECT TO_NUMBER(price_str,
  '999.99') FROM imports;
```

Safe Conversions — TRY_ Functions

KEY IDEA: TRY_ functions work like CAST/CONVERT but return **NULL** on failure instead of throwing an error. Great for dirty data!

TRY_CAST()

TRY_CAST(expr AS data_type)

✓ SQL Server only

```
SELECT TRY_CAST('abc' AS INT);
-- Returns: NULL (no error!)
SELECT TRY_CAST('42' AS INT);
-- Returns: 42
```

TRY_CONVERT()

TRY_CONVERT(data_type, expr [,style])

✓ SQL Server only

```
SELECT TRY_CONVERT(DATETIME,
  'not-a-date');
-- Returns: NULL
SELECT TRY_CONVERT(DATETIME,
  '2025-03-15');
```

TRY_PARSE()

TRY_PARSE(string AS type USING culture)

✓ SQL Server only

```
SELECT TRY_PARSE('15/03/2025'
  AS DATE USING 'en-GB');
-- Returns: 2025-03-15
SELECT TRY_PARSE('invalid'
  AS DATE USING 'en-US');
-- Returns: NULL
```